포트폴리오

이영주 · 대용량 렌더링 최적화와 앱 출시·운영 경험을 가진 5년차 프론트엔드 개발자

epww1502@gmail.com github.com/epww1502 이력서 → 경력기술서 →

10,000 → 100ms
대용량 차트 렌더링을 계열 단위로 바꿔 100배 단축

3개 플랫폼 · 5회
iOS·Android·웹을 한 코드로, 4개월간 정식 릴리스 5회

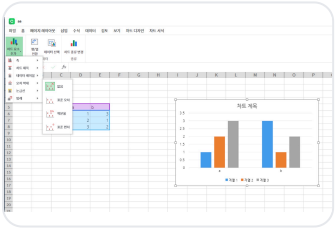
401 갱신 1회
동시에 만료된 요청들을 한 번의 토큰 갱신으로 복구

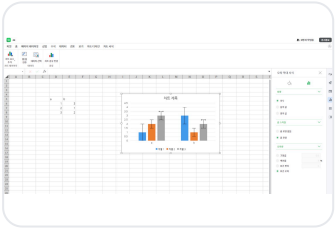
티맥스가이아 · 2022.01 – 2025.01

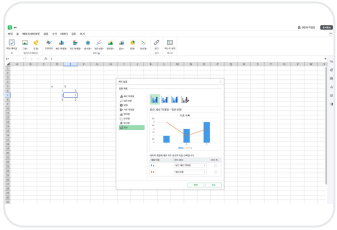
GAIA OFFICE Chart

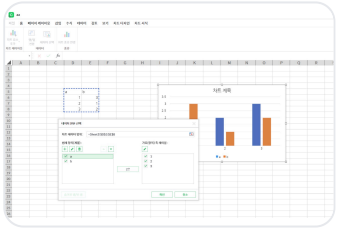
웹오피스의 차트를 Victory 라이브러리 위에 만들었습니다. 2022년 9월 팀 개편 이후에는 차트 기능 전체의 개발과 유지보수를 책임졌고, 신규 입사자 멘토링도 맡았습니다.

역할	스택	성과
차트 모델·컴포넌트 설계, 기능 전체 개발·유지보수	React, TypeScript, MobX, Victory	대용량 렌더링 10,000ms → 100ms


차트 요소 추가 메뉴


오차 막대 서식


차트 종류 변경 · 콤보 차트


데이터 선택 다이얼로그

대용량 차트 렌더링 개선 (2023.01 – 03)

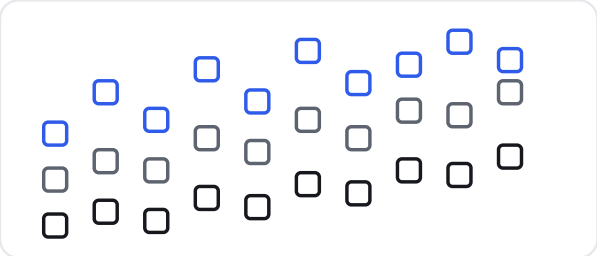
문제 데이터가 많은 차트일수록 급격히 느려져, 100×100 차트 하나를 그리는 데 약 10초가 걸렸습니다. 30×30만 넘어도 삽입과 선택, 크기 조절이 느렸고, 서버로 가는 메시지가 몰려 데드락이 생기기도 했습니다.

원인 가장 작은 단위인 데이터 포인트마다 컴포넌트와 모델을 만들고 있어서, 100×100 차트면 컴포넌트가 10,000개 생겼습니다.

해결 Victory가 포인트마다 계산하는 path를 계열(series) 단위로 모아, 계열 하나를 path 하나로 그리도록 바꿨습니다. 데이터 포인트는 모델의 자식으로 만들지 않고 목록으로 관리해 서버로 보내는 메시지 수도 줄였습니다. 오차 막대, 데이터 레이블, 선택, 행/열 전환, 타입 변경까지 모두 새 구조에 맞췄습니다.

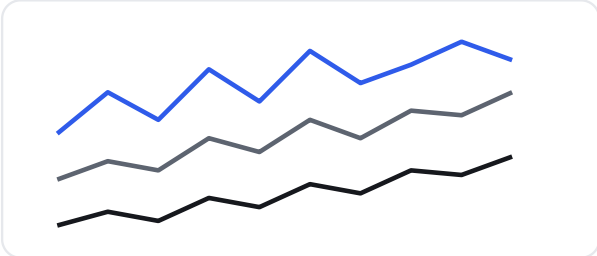
결과 **100×100 차트 렌더링 10,000ms → 100ms, 30×30 차트 삽입 11초 → 2초, 서버 데드락 해결**

변경 전 · 포인트마다 컴포넌트



100×100 차트 = 컴포넌트 10,000개

변경 후 · 계열마다 path 하나



100×100 차트 = 컴포넌트 100개

네모 하나가 컴포넌트 하나입니다. 변경 후에는 계열 하나가 선 하나로 그려집니다.

```
// 변경 전: 포인트마다 컴포넌트를 만든다
series.points.map((point) => <DataPoint key={point.id} model={point} />);

// 변경 후: 계열의 포인트 path를 모아 한 번에 그린다
const d = series.points.map(toPath).join(' ');
<path d={d} />;
```

실제 코드가 아니라 구조만 단순화한 예시입니다.

Excel과 같은 오차 막대 구현 (2022.03 – 05)

문제 Excel과 같은 오차 막대가 필요했지만, Victory는 cap 없는 형태나 한 축만 보이는 형태를 지원하지 않았습니다. 막대형 차트에서는 계열 사이 간격 때문에 위치도 어긋났습니다.

해결 고정값, 백분율, 표준오차, 표준편차 4종을 계열 모델 안에 두고, 차트 위에 별도 그룹으로 겹쳐 그렸습니다. Victory의 Error Bar가 만드는 path 값을 받아 수정해서 그렸고, 오차량 계산식도 구현했습니다.

결과 MS Excel 2021과 같은 값과 크기로 표시

그 외 맡은 일

- Chart 모델·컴포넌트 트리·데이터 관리 구조 설계, OOXML 기준 Import/Export
- 차트 종류 변경과 행/열 전환. 모델을 복제·전치해 오차 막대, 데이터 레이블, 스타일을 유지하고 Undo/Redo 지원
- 데이터 참조를 문자열에서 셀 주소 객체로 바꾸고, 여러 범위 선택(Multi-Range)과 다단계 항목(Multi-Level Category) 지원
- 데이터 선택 다이얼로그. 범위를 바꾸면 미리보기에 바로 반영하고, 범위가 맞지 않으면 오류를 띄움
- 미니 셀. Word·Point 문서에 차트를 넣기 위해 데이터용 셀 문서를 링크 방식으로 연결하는 구조 설계

관악연구소 · 2025.12 – 현재

GLAB · AI 디지털 PB 앱

개인 투자자에게 AI 투자 상담을 제공하는 앱으로, 2026년 7월 정식 출시 후 머니투데이에 소개됐습니다.
프론트엔드 파트장으로 기획과 디자인 검토부터 빌드, 배포, 운영까지 맡았고, 코드리뷰와 스토어 출시, AWS 웹

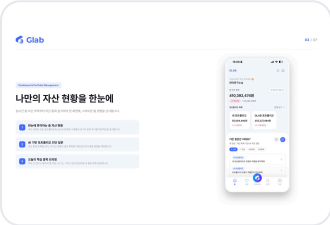
인프라와 배포 자동화를 담당했습니다.

역할
프론트엔드 파트장
기획 · 빌드 · 배포 · 운영

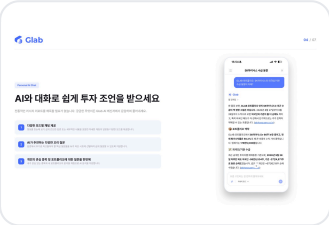
기여
팀 기능 PR 약 540개 중 278개 작성
동료 PR 149건 리뷰·머지

서비스
App Store · Google Play
glab.gwanaklab.com (웹·PWA) ·
머니투데이 보도

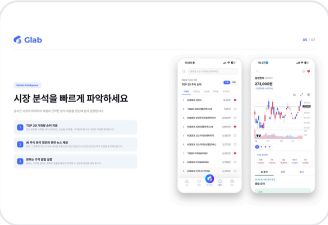
스택
Expo, React Native, TanStack Query, Zustand, NativeWind, Firebase Auth



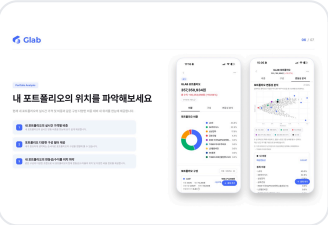
홈 · 자산 현황



AI 채팅



탐색 · 실시간 랭킹



포트폴리오 분석

AI 채팅 스트리밍 렌더링 최적화

문제 답변이 스트리밍되는 동안 토큰이 올 때마다 store와 채팅 화면 전체가 다시 그려졌고, 다른 대화가 스트리밍 중일 때도 보고 있는 화면이 다시 그려졌습니다.

원인 이미 있던 throttle은 대기 값이 할당되지 않아 실제로는 동작하지 않았고, 대화 목록 구독이 채팅 화면 전체에 걸려 있었습니다.

해결 스트리밍 중에는 store 동기화를 건너뛰고, 메시지 컴포넌트 비교 함수로 마지막 메시지만 다시 그리게 했습니다. 대화 목록 구독은 사이드바로 옮겼습니다. 채팅 화면은 탭 밖 별도 스택으로 올리고 `freezeOnBlur`를 적용해, 보이지 않을 때는 상태를 유지한 채 렌더링만 멈추게 했습니다.

```
// 내용이 바뀐 메시지만 다시 그린다. 스트리밍 중에는 마지막 메시지만 해당된다.
const Message = memo(MessageView, (prev, next) =>
  prev.message.id === next.message.id && prev.message.text === next.message.text,
);

// 채팅은 탭 밖 스택에 두고, 보이지 않는 동안에는 렌더링을 멈춘다.
<Stack screenOptions={{ headerShown: false, freezeOnBlur: true }} />;
```

실제 코드가 아니라 구조만 단순화한 예시입니다.

OTA 업데이트와 웹 배포 자동화

상황 작은 팀이 iOS·Android·웹을 동시에 운영해야 했고, 스토어 심사를 거치면 작은 수정도 바로 내보낼 수 없었습니다.

판단 Expo 한 코드베이스로 세 플랫폼을 만들고, 화면·로직 수정은 EAS Update(OTA)로 바로 내보내기로 했습니다.

구현 업데이트는 백그라운드에서 미리 받아 두었다가 앱으로 돌아올 때 적용해 사용 중 화면이 깜빡이지 않게 했고, 복귀할 때의 확인 요청에는 15분 간격을 두었습니다. 네이티브 버전이 다른 앱에는 업데이트를 내려보내지 않아, 옛 앱이 새 네이티브 코드를 찾다가 종료되는 문제를 막았습니다. 웹은 도메인, S3, CloudFront로 AWS 인프라를 구성했습니다. 해시가 붙은 파일은 길게 캐시하고 HTML과 서비스 워커는 매번 확인하도록 나눠, 배포 후에도 옛 화면이 남던 문제를 해결했습니다.

결과 **v1.1 → v1.5까지 4개월간 정식 릴리스 5회, 그 사이 수정은 심사 없이 OTA로 반영**



개발·운영·디버그 브랜치마다 같은 흐름으로 배포됩니다.

소셜 로그인과 Android의 Apple 계정 연결 해제

상황 Kakao·Google·Apple 로그인을 지원해야 했고, App Store는 탈퇴할 때 Apple 계정 연결까지 해제하도록 요구합니다. 그런데 Android에는 Apple 로그인 SDK가 없습니다.

구현 Kakao SDK와 Firebase Auth(Google·Apple)로 로그인을 구성하고, 탈퇴 시 각 IdP의 토큰·코드를 서버에 함께 넘겨 연결을 해제했습니다. Android에서 Apple 인증 창을 띄우면 App Links 없이는 리다이렉트를 기다리다 창이 닫혔고, 쿼리로는 id_token을 받을 수 없었습니다. 그래서 웹 페이지에서 Apple JS로 code와 id_token을 한 번에 받아 `glab://` 딥링크로 앱에 넘기는 브릿지를 만들었습니다.

결과 **세 플랫폼 모두 같은 로그인·탈퇴 흐름으로 스토어 심사 기준 충족**

그 외 맡은 일

- 파트장 업무로 프론트 기획과 디자인 검토, 출시 전 버튼 위치·화면 흐름 점검, 전체 커밋 검토와 코드리뷰(동료 PR 264건 중 149건 머지), PR 템플릿 도입, 스토어 출시·OTA 업데이트 관리
- 디자인 산출물 없이 시작해 공통 컴포넌트와 테마·컬러 토큰을 먼저 만들고 화면을 그 위에 구성
- 실시간 랭킹에서 연결 전에 보낸 구독 해지가 늦게 도착해 로딩 화면에 멈추던 경쟁 조건 수정, 1초 안에 데이터가 없으면 다시 구독
- 실시간 시세 숫자 애니메이션 최적화. 자리마다 29개씩 그리던 행을 바뀌는 구간만 그리게 줄이고(1 차이일 때 3개), 간격을 서버 시세 주기에 맞춤
- 홈 개편(관심 종목 접기, 뉴스 필터, 탭 다시 누르면 맨 위로), 추천 질문을 채팅에서 홈으로 이동, 관심 등록 캐시 정합성
- 포트폴리오 매수·매도·편집·복사와 도넛·샤프 차트
- 로그인 세션 정합성. 서버 로그아웃이 실패하면 로컬 토큰을 지우지 않고 재시도를 안내하고, 로그인 직후 사용자 조회가 실패하면 서버 세션까지 정리
- 다국어(i18n) 체계 도입. 하드코딩된 문구를 한국어·영어 리소스로 옮기고, 기기 언어 감지와 개발 중 누락 키 표시·eslint 검사 추가

- 운영·개발·디버그 앱 환경 분리(앱 설정, EAS 프로필, Firebase 설정, 배포 대상)
- PostHog 분석, PWA, 모바일 웹에서 스토어 설치 유도, 점검 모드
- 앱 만족도 피드백 팝업. 설치 후 3일, 사용 2일이 지난 사용자에게 AI 답변 평가나 채팅 5회 같은 시점에 띄우고, 접수된 피드백은 관리자 웹 고객 지원 화면에서 처리

관악연구소 · LG CNS 고객사 프로젝트 · 2025.01 – 2025.11

LG CNS COP · 생성형 AI 이미지 편집 플랫폼

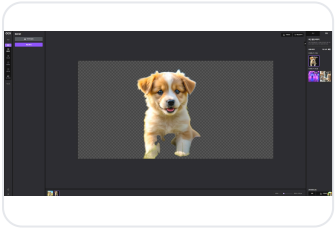
텍스트로 이미지를 만들고 배경 확장·제거, 부분 편집, 업스케일링, 이미지 속 텍스트 편집까지 하는 웹 서비스입니다. AI 페스타에서 공개돼 언론에 보도됐고, 산돌구름 AI Studio 베타로 오픈했습니다. ADMIN 프론트엔드 전체를 설계·구현했고, 서비스 프론트엔드에서는 공통 컴포넌트·인증 구조와 개발·배포 자동화를 구축했습니다.

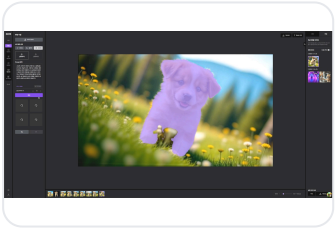
역할
ADMIN 프론트엔드 설계·구현
서비스 공통 기반·배포 자동화

팀 구성
프론트엔드 3명, 백엔드 2명

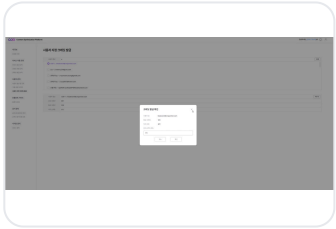
공개
AI 페스타 · 디지털타임스 보도
산돌구름 AI Studio 베타 · 동작 영상

스택
React, TypeScript, TanStack Query·Table, Zustand, Zod, Tailwind


배경 제거


부분 편집 · 영역 선택


ADMIN · 사용 현황

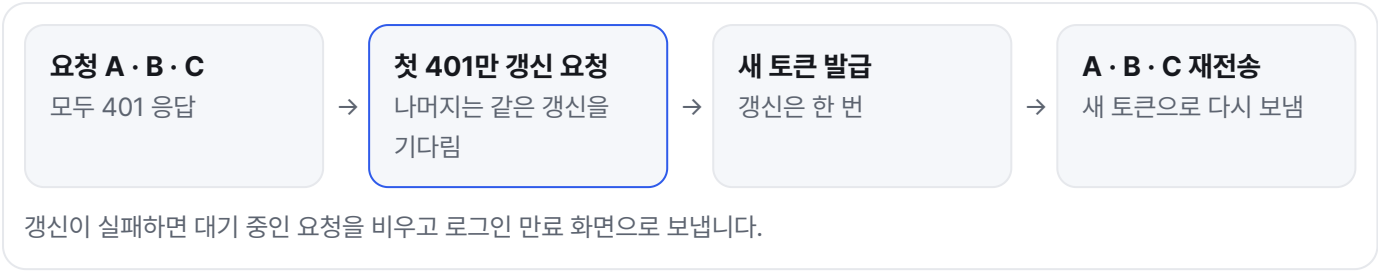

ADMIN · 크레딧 발급

토큰 만료 시점에 동시에 발생하는 401

문제 토큰이 만료되는 순간 이미지 생성·편집 요청이 여러 개 나가면 한꺼번에 401을 받았습니다. 그러면 갱신 요청이 중복되거나, 옛 토큰으로 다시 요청하거나, 실패한 요청이 그대로 유실됐습니다. 내부망 인증 서버와 브라우저 호출이 맞물려 CORS·쿠키 처리도 프론트에 몰려 있었습니다.

해결 refresh token 재발급은 백엔드로 옮겨 인증 흐름을 하나로 만들었습니다. 프론트에서는 공통 인터셉터가 401을 모아 갱신을 한 번만 실행하고, 기다리던 요청은 새 토큰으로 다시 보냅니다. 갱신이 실패하면 대기 요청을 비우고 로그인 만료 상태로 전환합니다.

결과 **동시 요청 상황에서도 인증 실패 복구가 일관되게 동작, 서비스와 ADMIN에 같은 구조 적용**



갱신이 실패하면 대기 중인 요청을 비우고 로그인 만료 화면으로 보냅니다.

```
let refreshing: Promise<void> | null = null;

async function onUnauthorized(request: Request) {
  // 이미 갱신 중이면 그 갱신을 함께 기다린다. 갱신 요청은 한 번만 나간다.
  refreshing ??= refreshToken().finally(() => {
    refreshing = null;
  });

  try {
    await refreshing;
  } catch {
    return expireSession(); // 갱신 실패: 대기 요청을 버리고 로그인 만료로 전환
  }
  return retry(request); // 새 토큰으로 다시 보낸다
}
```

실제 코드가 아니라 구조만 단순화한 예시입니다.

역할별 권한과 크레딧 발급 ADMIN

상황 운영자가 사용자에게 크레딧을 수동으로 발급하고 계정·그룹·공지·정책을 관리해야 했고, 역할마다 볼 수 있는 기능이 달라야 했습니다.

구현 역할 4종에 따라 메뉴·탭·접근 경로를 나누고, 크레딧 발급은 마스터에게만 보이게 했습니다. 발급량은 숫자만 받아 10~9,999 범위를 벗어나면 버튼을 막고, 발급 전에 내용을 확인 모달로 한 번 더 보여 줍니다. 사용 현황 필터는 URL에 담아 새로고침·뒤로 가기에도 유지되게 하고, 그룹에서 사용자로 내려가 보는 드릴다운과 엑셀 내보내기를 넣었습니다.

결과 **ADMIN 전체(사용 현황, 크레딧 발급, 계정·그룹, 공지, 정책)를 테스트 일정 안에 완료**

공통 오류 처리와 배포 자동화

상황 여러 명이 서비스 화면과 ADMIN을 동시에 만들어야 해서, 로그인·인증·오류 처리가 화면마다 달라질 위험이 있었습니다.

구현 로그인과 공통 UI 구조를 설계하고 팀 공통 컴포넌트를 정리했습니다. 서버 오류는 쿼리 캐시에서 한곳으로 모아 상태·서비스 코드별로 토스트를 띄우고, 응답은 Zod로 검증해 화면마다 따로 처리하지 않게 했습니다.

자동화 husky와 lint-staged로 커밋 전에는 lint, push 전에는 lint와 타입 검사를 돌려 깨진 코드가 올라가지 않게 했고, main에 push하면 GitHub Actions가 빌드, S3 배포, CloudFront 캐시 무효화까지 처리하도록 했습니다.

그 외 맡은 일

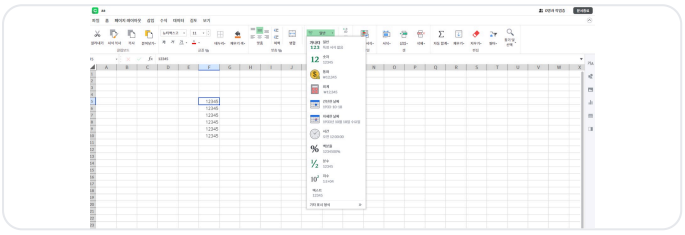
- 마이페이지·텍스트 편집 기능 구현과 세부 기획 참여, 테이블 편의 기능 제안
- DTO·서비스 문서를 작성해 백엔드와 API 형식 합의

티맥스가이아 · 2022.06 – 2025.01

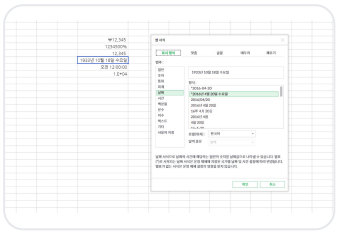
GAIA OFFICE Cell · 웹 스프레드시트

웹오피스의 스프레드시트입니다. 표시 형식, 피벗 테이블, 병합, 그래픽 요소를 개발했고, 2년 동안 이슈 약 555건을 처리했습니다(약 35%는 담당 기능 밖). NTIS 국가 R&D 과제(12859502)에 참여했습니다.

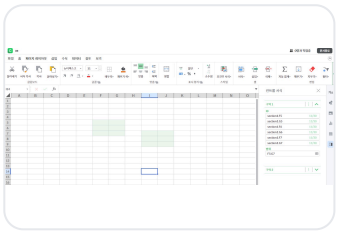
역할	스택	이슈 처리
Cell 기능 개발·유지보수	React, TypeScript, Canvas, Java	약 555건



홈 탭 · 표시 형식



셀 서식 · 표시 형식



문서 자동 완성 · 구역 설정

시트 전체를 병합하면 멈추는 문제 (2024.03 – 05)

문제 시트 전체(약 170억 셀)나 행·열 전체를 병합하면 오피스가 멈췄습니다.

원인 병합 전 값을 백업할 때 범위 안의 데이터 벡터를 빈 칸까지 전부 복사하고 있었습니다.

해결 빈 데이터는 건너뛰고, Dimension Reference로 실제 데이터가 있는 최소 범위만 복사하도록 바꿨습니다. 셀마다 만들던 서버 메시지는 범위 단위로 묶고, 하나로 섞여 있던 병합·병합 해제 명령을 분리했습니다. 셀·행/열·전체 선택에 따라 스타일 적용 범위도 다시 계산하고, Excel 2021의 병합 스타일 규칙을 분석해 맞췄습니다.

결과 **멈추던 전체 병합을 2초 안에 처리, 넓은 범위 병합 평균 10초 → 1.5초**

변경 전
선택 범위 전체를 빈 칸까지 복사

→

빈 데이터 제외
값이 있는 셀만 대상

→

최소 범위 계산
실제 데이터가 있는 영역

→

변경 후
그 범위만 복사해 병합

복사 대상을 선택 범위 전체에서 실제 데이터 범위로 줄였습니다.

명령마다 반복되던 Canvas 렌더링

문제 명령 하나가 실행될 때마다 Canvas 갱신을 호출해서, 병합이나 스타일 적용처럼 여러 명령이 이어지는 동작에서는 같은 범위를 여러 번 다시 그렸습니다.

해결 명령이 도는 동안에는 갱신할 범위와 조건만 모아 두고, 명령이 끝나는 시점에 한 번만 그리게 했습니다. 처음에는 지연 호출(debounce)을 썼지만 빠르게 입력하면 화면이 멈춘 듯 보여서, 명령 종료 시점에 그리는 방식으로 바꿨습니다.

결과 **병합·스타일 적용 시 렌더 호출 5회 → 1회**

한국 서식을 지원하는 표시 형식(Number Format) (2023.02 – 05)

문제 numfmt 라이브러리를 썼지만 한국 날짜, 경과 시간(`[h]:mm:ss`), 원화 기호, 소수점 증감은 제대로 처리하지 못했습니다.

해결 입력값을 라이브러리가 이해하는 형식으로 바꾸는 변환 로직을 규칙마다 구현했습니다. 회계 서식은 Canvas `measureText` 로 글자 폭을 재서 통화 기호와 숫자 사이를 셀 너비에 맞게 띄웠습니다.

문서 자동 완성 TFT (2023.06 – 08)

- 외부 앱 데이터(사고 접수 등)를 Cell 문서의 지정 구역에 자동으로 채워 문서를 만드는 기능. TFT에서 가장 낮은 연차로 대표 기능을 담당
- Word·Point·Cell 공통 구역 관리 구조와 DB 테이블 설계, MERGE INTO로 중복 태그 Insert/Update를 한 번에 처리
- 배달서비스공제조합 서비스에 제공

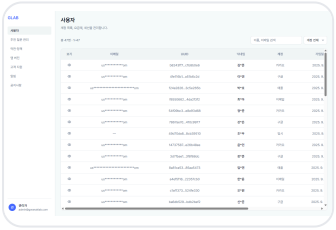
관악연구소 · 2026.08 – 현재

GLAB 관리자 웹

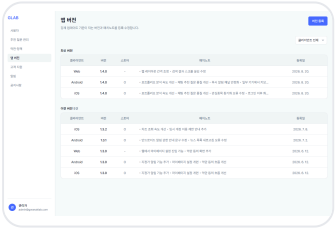
GLAB 앱 운영을 위한 관리자 웹입니다. 사용자, 공지, 약관, 앱 버전, 추천 질문, 고객 지원까지 21개 화면을 설계·구현하고 실제 API와 연동해 운영에 쓰고 있습니다.

역할
기획·설계·구현

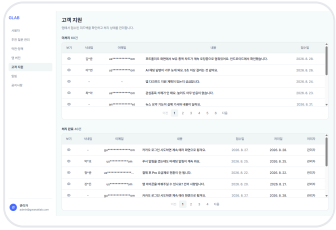
스택
Vite, React 19, React Router 7, Zustand, React Hook Form, Zod, TipTap



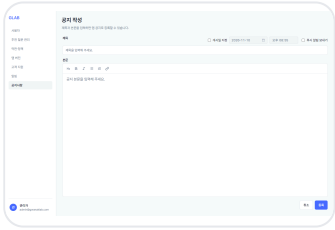
사용자 · 개인정보 마스킹



앱 버전 · 최신/이전 분리



고객 지원 · 처리 상태 관리



공지 작성 · 예약 게시·푸시

운영 규칙을 담은 화면과 입력 검증

상황 관리자 웹에서 바꾼 공지·약관·앱 버전은 그대로 앱 사용자에게 나갑니다.

판단 운영 중 실수가 나기 쉬운 지점을 먼저 정리해 화면과 입력 검증에 반영했습니다. 목 데이터로 흐름을 확인한 뒤 실제 API에 연결했습니다.

구현 공지는 지난 시각으로 예약할 수 없고, 이미 게시된 공지는 게시일과 푸시 설정을 바꿀 수 없게 했습니다. 사용 중인 약관 유형은 삭제를 막고, 앱 버전은 iOS·Android·웹별로 등록합니다. 버전 목록은 최신/이전 테이블을 나눠 지금 배포 중인 버전이 묻히지 않게 했습니다. 규칙은 Zod 스키마로 옮겨, 작성과 수정에 다른 규칙을 적용하고 약관 본문은 HTML 안에 실제 글자가 있는지까지 검사했습니다.

```
// 예약 게시는 현재 시각 이후만 받는다. 이미 게시된 공지를 고칠 때만 지난 시각을 그대로 둔다.
const noticeSchema = (options: { allowPast: boolean }) =>
  baseSchema.superRefine((value, ctx) => {
    if (!value.scheduled || options.allowPast) return;
    if (isPast(value.publishAt)) {
      ctx.addIssue({ code: 'custom', path: ['publishAt'], message: '현재 시각 이후로 선택해 주세요.' });
    }
  });
```

실제 코드가 아니라 구조만 단순화한 예시입니다.

에디터 재사용과 개인정보 마스킹

문제 관리자 웹에서 쓴 글이 앱에서 그대로 보여야 하고, 운영자 화면에 사용자 정보가 그대로 노출되면 안 됩니다.

구현 TipTap 에디터 하나를 출력 형식만 바꿔 재사용해, 공지는 Markdown, 약관은 앱의 HTML 렌더러에 맞춘 HTML로 저장하게 했습니다. 사용자 목록과 고객지원 화면에서는 이메일·닉네임을 기본으로 가리고, 필요한 행 하나만 열어 볼 수 있게 했습니다.

그 외 맡은 일

- 목록 화면은 제목·검색·필터와 표 머리글을 고정하고 행만 스크롤. 검색은 버튼을 눌렀을 때만 제목·본문 조회
- 고객 지원은 미처리/처리 완료 목록을 나누고 처리일·처리자 표시
- 브라우저 기본 영문 검증 대신 해당 입력 아래에 한국어 오류 표시

개인 프로젝트 · 2026.10 – 현재 · iOS 출시, Android 출시 예정

DoIt2 · 함께 쓰는 버킷리스트 앱

연인·친구·가족이 공간을 만들어 가고 싶은 곳과 해보고 싶은 것을 함께 모으고, 다녀온 뒤 사진과 후기를 남기는 앱입니다. 기획, 앱, Supabase 백엔드, 스토어 등록과 App Store 심사 대응까지 2주 반 동안 진행했습니다 (커밋 80개).

역할
기획·설계·개발

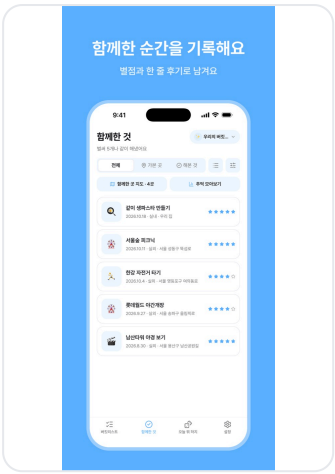
규모
DB 마이그레이션 32개, RLS 정책 63개
Edge Function 5개, 테스트 14개 파일

스택
Expo SDK 58, Expo Router, NativeWind, Zustand, Supabase, Jest

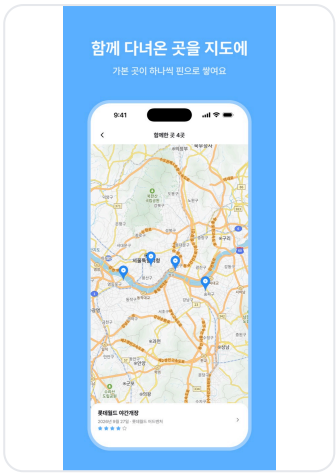
코드
github.com/epww1502/DoIt2



버킷리스트



함께한 기록



다녀온 곳 지도



추억 돌아보기

여러 명이 동시에 수정할 때의 동기화

문제 한 공간을 여러 명이 같이 쓰기 때문에, 두 사람이 동시에 체크리스트를 고치면 한쪽 변경이 덮어써질 수 있었습니다.

해결 목록 전체를 덮어쓰지 않고 체크, 추가 같은 변경 단위로 기록해 서버의 최신 상태 위에 다시 적용했습니다. 같은 변경이 두 번 전송돼도 결과가 달라지지 않고, 오프라인일 때는 변경을 쌓아 두었다가 연결되면 보냅니다.

검증 두 사람이 서로 다른 항목을 체크해도 둘 다 남는지, 같은 변경을 두 번 보내도 결과가 같은지를 Jest 테스트로 작성했습니다.

```
it('두 사람이 다른 할 것을 체크해도 둘 다 남는다', () => {
  const server = [todo('a'), todo('b')];
  // 나는 a, 상대는 b를 체크 (어느 쪽이 먼저 반영되든 같은 결과)
  const result = applyTodoOps(
    applyTodoOps(server, [{ type: 'check', todoId: 'a', done: true }]),
    [{ type: 'check', todoId: 'b', done: true }],
  );
  expect(result.map((t) => t.done)).toEqual([true, true]);
});

it('같은 변경을 두 번 보내도 결과가 같다', () => {
  const ops = [{ type: 'check', todoId: 'a', done: true }, { type: 'add', todo: todo('c') }];
  const once = applyTodoOps([todo('a')], ops);
  expect(applyTodoOps(once, ops)).toEqual(once);
});
```

저장소의 `__tests__/todoOps.test.ts` 에서 가져와 줄였습니다.

RLS와 Edge Function으로 권한 관리

상황 같은 공간 사람끼리만 데이터를 봐야 하는데, 앱 코드만으로 막으면 요청을 직접 보내는 경우를 막을 수 없습니다.

구현 모든 표에 RLS를 걸어 공간 멤버만 읽고 쓰게 하고, 공간 참여·공간장 위임처럼 여러 표를 바꾸는 작업은 DB 함수로 묶었습니다. 카카오 API 키는 Edge Function 프록시 뒤에 두고, 링크 미리보기는 내부망 주소를 리다이렉트 이후까지 막았습니다. 푸시 알림과 계정 삭제도 서버 함수로 처리했습니다.

심사 대응 App Store 심사에서 신고·차단 기능(가이드라인 1.2)을 요구해, 신고는 쓰기만 가능하고 차단한 사람의 댓글은 보이지 않도록 RLS로 추가했습니다.

업데이트, 오류 수집, 사진 캐시

구현 스플래시 동안과 앱으로 돌아올 때 OTA 업데이트를 받고, 서버의 최소 버전보다 낮은 앱은 업데이트 화면으로 막아 서버에 저장된 스토어 주소로 보냅니다. 오류는 기기에 먼저 저장했다가 서버로 보내 앱이 종료돼도 기록이 남게 했고, 화면에서 오류가 나면 앱이 꺼지는 대신 다시 시도 화면을 보여 줍니다.

성능 사진은 올리기 전에 줄이고, 매번 바뀌는 임시 링크 대신 저장소 경로를 기준으로 기기에 캐시해 한 번 본 사진은 다시 받지 않게 했습니다. 캐시에 없는 사진의 링크는 모아서 한 번에 요청합니다.

개인 프로젝트 · 2026.10 – 현재

Month · 복약 알림 앱

먹을 약과 시간을 등록하면 하루 복용 일정을 보여 주고, 알림과 홈 화면 위젯에서 바로 복용 여부를 체크하는 iOS·Android 앱입니다. 서버 없이 기기에만 저장합니다. Figma 스토리보드를 바탕으로 설계·개발했습니다.

역할	플랫폼	스택
설계·개발	iOS · Android	Expo SDK 58, Expo Router, expo-widgets, Kotlin, expo-sqlite, Zustand, Jest

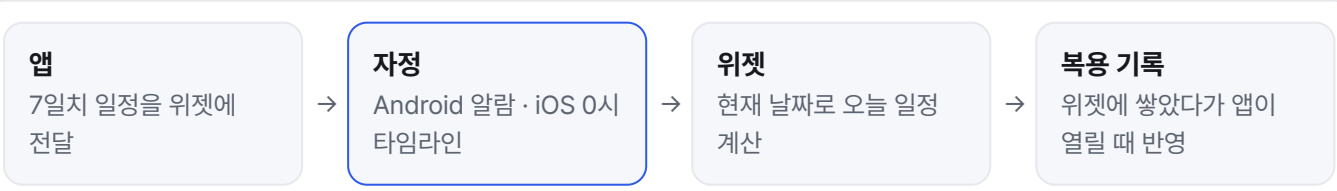
앱을 열지 않아도 날짜가 바뀌는 위젯

상황 위젯은 앱이 넘겨 준 값만 그립니다. 그래서 앱을 열지 않으면 자정이 지나도 어제 일정이 그대로 남았고, 위젯에서 누른 "복용완료!"도 앱에 반영되지 않았습니다. Android용 expo-widgets에는 위젯이 저장한 값을 JS에서 읽는 API가 없었습니다.

판단 하루치가 아닌 7일치 일정을 위젯에 넘기고, 오늘 보여 줄 화면은 위젯이 그려질 때 현재 날짜로 스스로 계산하게 했습니다.

구현 Android는 Kotlin 네이티브 모듈 `widget-bridge`를 만들어 위젯 저장값을 읽고, 자정 알람으로 위젯을 다시 그리게 했습니다. 재부팅하거나 시간·시간대가 바뀌면 알람을 다시 겁니다. iOS는 매일 0시 타임라인 항목으로 처리했습니다. 위젯 버튼 기록은 위젯 쪽에 쌓아 두었다가 앱이 열릴 때 반영하고, 라이브러리 수정은 patch-package 패치로 관리합니다.

검증 iOS·Android 위젯 코드를 실제로 변환해 실행하는 테스트를 만들어, 두 플랫폼의 화면 계산 결과가 같은지 확인합니다.



앱이 꺼져 있어도 위젯이 날짜에 맞는 일정을 스스로 고릅니다.

iOS·Android 알림 예약 제한 대응

상황 iOS는 예약 알림이 64개로 제한되고, Android 14부터는 새로 설치하면 정확한 알람 권한이 꺼져 있어 알림이 몇 분씩 늦을 수 있습니다.

구현 반복 알림 대신 앞으로 7일치를 날짜별로 예약하고, 앱을 열 때와 기록이 바뀔 때마다 다시 채웁니다. 같은 시간의 약은 알림 하나로 묶었습니다. Android는 알림을 켤 때 권한을 안내한 뒤 시스템 설정으로 보내고, 거부해도 알림은 켠 채 저장합니다.

그 외 작업

- Jest 단위 테스트 66개(일정 계산·완료율·알림 예약·위젯), 커밋·push 때 husky 검사, PR마다 GitHub Actions CI
- Windows 로컬 빌드의 경로 길이 문제를 해결하는 Expo 설정 플러그인